

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ,
МЕХАНИКИ И ОПТИКИ»**

Факультет безопасности информационных технологий

Дисциплина:
«Алгоритмы и структуры данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

Топологическая сортировка по схеме Тарьяна

Выполнил:
Студент гр. N3248
Жук Анастасия Валерьевна

Подпись: _____

Проверил:
Ерофеев Сергей Анатольевич

Подпись: _____

Санкт-Петербург
2025 г.

Техническое задание

Разработать программу топологической сортировки по схеме Тарьяна, записывая числа из файла в статический или динамический массив по выбору пользователя.

Переменные

Константы (#define)

Название переменной	Расшифровка	Тип	Диапазон	Емкость, байт
WHITE	Вершина не посещена (для обхода в глубину)	int	от -2 147 483 648 до 2 147 483 647	4
GRAY	Вершина в процессе обработки (для обхода в глубину)	int	от -2 147 483 648 до 2 147 483 647	4
BLACK	Вершина полностью обработана (для обхода в глубину)	int	от -2 147 483 648 до 2 147 483 647	4
MAX_STATIC_SIZE	Максимальный размер статической матрицы	int	от 0 до 2 147 483 647	4
INITIAL_BUF_SIZE	Начальный размер массива для чтения строк из файла	int	от 0 до 2 147 483 647	4

Локальные переменные (основные, в функции main)

Название переменной	Расшифровка	Тип	Диапазон	Емкость, байт
choice	Выбор типа массива (1 — статический, 2 — динамический)	int	от -2 147 483 648 до 2 147 483 647	4
rows, cols	Размеры матрицы (строки и столбцы)	int	от 0 до 2 147 483 647	4

line	Динамически выделенная строка для чтения из файла	char*	Любой символ	1 * len(line)
matrix	Матрица смежности (статическая или динамическая)	double[][] или double**	+/- 1.7 * 10 ⁻³⁰⁸ до 1.7 * 10 ³⁰⁸ (1 double)	rows * cols * 8
colors	Массив цветов вершин для DFS (0 — белый, 1 — серый, 2 — черный)	int*	от -2 147 483 648 до 2 147 483 647 (1 int)	n * 4
result	Массив для хранения результата топологической сортировки	int*	от 0 до 2 147 483 647 (1 int)	n * 4
has_cycle	Флаг наличия цикла в графе	bool	True или False	1
index	Текущий индекс в массиве result	int	от 0 до 2 147 483 647	4
v, u, i, j	Вспомогательные переменные-счётчики	int	от 0 до 2 147 483 647	4

Переменные в функции is_valid_double

Название переменной	Расшифровка	Тип	Диапазон	Емкость, байт
str	Входная строка для проверки на число	const char*	Любой символ	1 * len(str)
endptr	Указатель на первый символ, не являющийся частью числа	char*	Любой символ	1
errno	Код ошибки (проверка на переполнение)	int	от -2 147 483 648 до 2 147 483 647	4

Переменные в функции read_dynamic_line

Название переменной	Расшифровка	Тип	Диапазон	Емкость, байт
buf_size	Текущий размер буфера	size_t	от 0 до 2^{64}	8
buffer	Буфер для хранения строки из файла	char*	Любой символ	buf_size * 1
pos	Текущая позиция в буфере	size_t	от 0 до 2^{64}	8
c	Символ из файла (код ASCII)	int	от 0 до 2 147 483 647	4
new_buf	Буфер, который создается для того, чтобы проверить, сможем ли мы выделить память для buffer нового размера buf_size	char*	Любой символ	buf_size * 1

Переменные в функциях dfs_static, dfs_dynamic, topological_sort_static и topological_sort_dynamic

Название переменной	Расшифровка	Тип	Диапазон	Емкость, байт
v	Текущая вершина для обработки	int	от 0 до 2 147 483 647	4
u	Вспомогательная переменная, с помощью которой производится обход в глубину из вершины v	int	от 0 до 2 147 483 647	4
n	Количество вершин в графе	int	от 0 до 2 147 483 647	4
matrix	Матрица смежности	double[][] или double**	+/- $1.7 * 10^{-308}$ до $1.7 * 10^{308}$ (1 double)	n * n * 8
colors	Массив цветов вершин	int*	от -2 147 483 648 до 2 147 483 647 (1 int)	n * 4
result	Результат сортировки	int*	от 0 до 2 147 483 647 (1 int)	n * 4
index	Текущий индекс в result	int*	от 0 до 2 147 483 647 (1 int)	4
has_cycle	Флаг наличия цикла	bool*	True или False	1

Функциональные маршруты

Перед тем как приступить к самой сортировке, необходимо считать из файла матрицу смежности или весов графа. Для топологической сортировки достаточно только матрицы смежности (матрицы, у которой на пересечении i -ой строки и j -ого столбца стоит 1, если существует дуга из вершины i в вершину j , и 0 в противном случае), но также можно рассмотреть матрицу весов, заменяя вес дуги из вершины i в вершину j на 1. Веса никак не влияют на сортировку, поэтому можем от них избавиться и перейти к матрице смежности.

Итак, для того, чтобы перейти к сортировке, нужно проверить следующие свойства матрицы:

1. Матрица состоит из численных значений, не содержит букв и слов;
2. Матрица не пустая;
3. Матрица квадратная, то есть количество столбцов в ней равно количеству строк;
4. В случае выбора пользователем статического массива, размеры считываемой матрицы не должны превышать заданных размеров заданного двумерного массива.

Если в строке присутствуют пустые строки и/или лишние пробелы, то в процессе считывания матрицы из файла в массив необходимо исключить лишние символы.

На первом этапе нужно проверить содержимое файла, чтобы определить, подходит ли оно нам: содержится ли там матрица и удовлетворяет ли она всем вышеописанным условиям. В процессе просмотра матрицы необходимо запомнить ее размеры, чтобы в дальнейшем можно было упростить выделение памяти для динамического массива.

Когда статический массив создан и для динамического выделена память, можно записать значения матрицы в них (по выбору пользователя). Если вдруг в файле лежала матрица весов графа, то превратим ее в матрицу смежности, заменив веса на единицы, которые подразумевают под собой наличие дуг между вершинами.

Когда матрица была успешно считана и записана в двумерный массив, можно переходить к самой топологической сортировке по схеме Тарьяна.

Топологическая сортировка — это процесс упорядочивания вершин ациклического ориентированного графа согласно частичному порядку, заданному рёбрами орграфа на множестве его вершин. Топологическую сортировку можно выполнить за $O(n)$ времени и памяти, если обойти все

вершины, используя поиск в глубину, и выводить вершины в момент выхода из неё — алгоритм разработан Робертом Тарьяном в 1976 году.

В самом начале все вершины белые. Для каждой из них осуществляется шаг алгоритма:

- если вершина чёрная, то ничего делать не надо;
- если вершина серая, то найден цикл, а значит топологическая сортировка невозможна;
- если вершина белая, то:
 - она красится в серый;
 - применяется шаг алгоритма для всех вершин, в которые можно попасть из текущей;
 - вершина красится в чёрный и помещается в окончательный список.

Полученный список, выведенный в обратном порядке, даст нам отсортированный порядок вершин графа.

Код на языке программирования C

```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <errno.h>

// Типы вершин для обхода в глубину
#define WHITE 0
#define GRAY 1
#define BLACK 2

#define MAX_STATIC_SIZE 100 // Максимальный размер статической матрицы
                             // (MAX_STATIC_SIZE x MAX_STATIC_SIZE)
#define INITIAL_BUF_SIZE 1000 // Размер динамического массива, в который
                              // считываются строки из файла

// Проверка: является ли строка корректным вещественным числом
// Возвращает 1 (True), если строка является числом, и 0 (False) в противном случае
int is_valid_double(const char *str) {
    char *endptr; // Указатель на первый символ, который не удалось преобразовать в число
    errno = 0; // Код ошибки (0 - все хорошо)
    strtod(str, &endptr); // Перевод строки в число

    // Проверяем два условия валидности:
    // 1. Что преобразование дошло до конца строки (\0) или до пробельного символа
    // 2. Что не произошло переполнения (errno не установлен в ERANGE)
    return (*endptr == '\0' || isspace(*endptr)) && errno != ERANGE;
}

// Считываем строку из файла. Так как она может быть любого размера, то делаем
// считывание в динамический массив
char* read_dynamic_line(FILE* file) {
    size_t buf_size = INITIAL_BUF_SIZE;
    char* buffer = malloc(buf_size); // Здесь будет лежать считанная из файла строка (одна
    // текущая)
    if (!buffer) return NULL; // Обработываем ошибку невозможности выделения памяти

    size_t pos = 0; // Позиция символа в массиве buffer
    int c; // Будет хранить код считываемого символа из таблицы ASCII

    while ((c = fgetc(file)) != EOF && c != '\n') {
        // Если нам не хватило места в массиве для всей строки, то нужно его увеличить
        if (pos + 1 >= buf_size) {
            buf_size *= 2;
            char* new_buf = realloc(buffer, buf_size);
        }
    }
}
```

```

    // Обрабатываем ошибку невозможности выделения памяти
    if (!new_buf) {
        free(buffer);
        return NULL;
    }
    buffer = new_buf;
}
buffer[pos++] = (char)c; // Положили в массив считанный символ
}

if (pos == 0 && c == EOF) {
    free(buffer); // Если строка пустая, то освобождаем выделенную память
    return NULL;
}

buffer[pos] = '\0'; // Добавляем конец строки
return buffer; // Возвращаем считанную строку
}

void dfs_static(int v, int n, double matrix[MAX_STATIC_SIZE][MAX_STATIC_SIZE], int*
colors, int* result, int* index, bool* has_cycle) {
    colors[v] = GRAY; // Красим рассматриваемую вершину в серый

    for (int u = 0; u < n; u++) {
        if (matrix[v][u]) {
            if (colors[u] == WHITE) { // Если можем попасть в белую вершину, то идем в нее
                (рекурсивный обход в глубину)
                dfs_static(u, n, matrix, colors, result, index, has_cycle);
            } else if (colors[u] == GRAY) { // Если нам попалась серая вершина, значит мы ее
                уже рассматривали, а значит в графе цикл
                *has_cycle = true;
            }
        }
    }
}

colors[v] = BLACK; // Красим рассмотренную вершину в черный, чтобы больше в нее не
заходить, так как все пути из нее мы рассмотрели
result[( *index)++] = v + 1; // Нумерация вершин с единицы
}

void topological_sort_static(int n, double matrix[MAX_STATIC_SIZE][MAX_STATIC_SIZE])
{
    int* colors = (int*)malloc(n * sizeof(int));
    int* result = (int*)malloc(n * sizeof(int));
    int index = 0;
    bool has_cycle = false;

    if (!colors || !result) {

```



```

    printf("Ошибка: не удалось выделить память\n");
    if (colors) free(colors);
    if (result) free(result);
    return;
}

for (int i = 0; i < n; i++) {
    colors[i] = WHITE; // Все вершины графа изначально белые (то есть подлежат
    рассмотрению)
}

for (int i = 0; i < n; i++) {
    if (colors[i] == WHITE) { // Осуществляем сортировку через обход в глубину
        dfs_static(i, n, matrix, colors, result, &index, &has_cycle);
    }
}

if (has_cycle) {
    printf("Ошибка: граф содержит циклы, топологическая сортировка невозможна\n");
} else {
    printf("Топологический порядок вершин: [");
    for (int i = n - 1; i >= 0; i--) {
        printf("%d", result[i]);
        if (i > 0) printf(", ");
    }
    printf("]\n");
}

free(colors);
free(result);
}

void dfs_dynamic(int v, int n, double** matrix, int* colors, int* result, int* index, bool*
has_cycle) {
    colors[v] = GRAY; // Красим рассматриваемую вершину в серый

    for (int u = 0; u < n; u++) {
        if (matrix[v][u]) {
            if (colors[u] == WHITE) { // Если можем попасть в белую вершину, то идем в нее
                (рекурсивный обход в глубину)
                dfs_dynamic(u, n, matrix, colors, result, index, has_cycle);
            } else if (colors[u] == GRAY) { // Если нам попалась серая вершина, значит мы ее
                уже рассматривали, а значит в графе цикл
                *has_cycle = true;
            }
        }
    }
}

```

```

    colors[v] = BLACK; // Красим рассмотренную вершину в черный, чтобы больше в нее не
    заходить, так как все пути из нее мы рассмотрели
    result[(*index)++] = v + 1; // Нумерация вершин с единицы
}

```

```

void topological_sort_dynamic(int n, double** matrix) {
    int* colors = (int*)malloc(n * sizeof(int));
    int* result = (int*)malloc(n * sizeof(int));
    int index = 0;
    bool has_cycle = false;

    if (!colors || !result) {
        printf("Ошибка: не удалось выделить память\n");
        if (colors) free(colors);
        if (result) free(result);
        return;
    }

    for (int i = 0; i < n; i++) {
        colors[i] = WHITE; // Все вершины графа изначально белые (то есть подлежат
        рассмотрению)
    }

    for (int i = 0; i < n; i++) {
        if (colors[i] == WHITE) { // Осуществляем сортировку через обход в глубину
            dfs_dynamic(i, n, matrix, colors, result, &index, &has_cycle);
        }
    }

    if (has_cycle) {
        printf("Ошибка: граф содержит циклы, топологическая сортировка невозможна\n");
    } else {
        printf("Топологический порядок вершин: [");
        for (int i = n - 1; i >= 0; i--) {
            printf("%d", result[i]);
            if (i > 0) printf(", ");
        }
        printf("]\n");
    }

    free(colors);
    free(result);
}

```

```

int main() {
    int choice;
    printf("В какой массив записать матрицу? (1 - статический, 2 - динамический): ");
    scanf("%d", &choice);
}

```

```

if (choice != 1 && choice != 2) {
    printf("Ошибка: некорректный тип массива\n");
    return 1;
}

const char *filename = "matrix.txt"; // Будем считывать только из одного файла

FILE* file = fopen(filename, "r");
if (!file) {
    perror("Ошибка: не удалось открыть файл");
    return 1;
}

// Первый проход для оценки содержимого файла и определения размеров матрицы
int rows = 0, cols = 0;
char* line = NULL;

while ((line = read_dynamic_line(file)) != NULL) {
    if (strlen(line) == 0) { // Встретили пустую строку
        free(line);
        continue;
    }

    int current_cols = 0; // Счетчик столбцов в текущей строке
    char *ptr = line; // Указатель для перемещения по строке
    char *token; // Указатель на начало текущего числа

    while (*ptr) { // Пока не конец строки
        // Пропускаем пробельные символы
        while (*ptr && isspace(*ptr)) ptr++;
        if (!*ptr) break; // Если строка закончилась, то выходим

        token = ptr; // Запоминаем начало числа
        while (*ptr && !isspace(*ptr)) ptr++; // Пропускаем все значимые символы (до конца
        числа)

        if (!is_valid_double(token)) { // Проверка, что считанный элемент матрицы не
        является числом
            printf("Ошибка: некорректное значение '%.*s'\n", (int)(ptr - token), token);
            free(line);
            fclose(file);
            return 1;
        }

        current_cols++;
    }

    if (cols == 0) cols = current_cols;
}

```

```

else if (current_cols != cols) {
    printf("Ошибка: разное количество элементов в строках\n");
    free(line);
    fclose(file);
    return 1;
}
rows++; // Одну строку обработали, идем к следующей
free(line);
}

if (rows == 0 || cols == 0) {
    printf("Ошибка: пустая матрица\n");
    fclose(file);
    return 1;
}

//Ситуация, когда рассматриваемая матрица больше по размеру, чем статический
массив
if ((choice == 1) && (rows > MAX_STATIC_SIZE || cols > MAX_STATIC_SIZE)) {
    printf("Ошибка: матрица слишком большая для статического массива\n");
    fclose(file);
    return 1;
}

if (rows != cols) {
    printf("Ошибка: матрица смежности должна быть квадратной\n");
    fclose(file);
    return 1;
}

rewind(file);

// Второй проход для записи данных в двумерный массив
if (choice == 1) { // Считываем в статический массив
    double matrix[MAX_STATIC_SIZE][MAX_STATIC_SIZE];
    for (int i = 0; i < rows; i++) {
        line = read_dynamic_line(file);
        while (strlen(line) == 0) line = read_dynamic_line(file);

        if (!line) {
            printf("Ошибка: не удалось прочитать строку %d\n", i + 1);
            fclose(file);
            return 1;
        }

        char *ptr = line;
        char *token;
        for (int j = 0; j < cols; j++) {

```

```

while (*ptr && isspace(*ptr)) ptr++; // Пропускаем пробельные символы

// Считываем число
token = ptr;
while (*ptr && !isspace(*ptr)) ptr++;

// Если пользователь ввел матрицу весов, то заменим веса на единицы, то есть на
обычные ребра (превратим матрицу весов в матрицу смежности)
double token_double = (double)strtod(token, NULL);
if (token_double == 0) matrix[i][j] = 0;
else matrix[i][j] = 1;
}
free(line);
}

// Вывод матрицы смежности графа
printf("Матрица %dx%d:\n", rows, cols);
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        printf("%g ", matrix[i][j]);
    }
    printf("\n");
}

int n = rows;
topological_sort_static(n, matrix);

} else { // Считывание в динамический массив
    // Выделяем память для динамического массива
    double** matrix = (double**)malloc(rows * sizeof(double*));
    for (int i = 0; i < rows; i++) {
        matrix[i] = (double*)malloc(cols * sizeof(double));
    }

    for (int i = 0; i < rows; i++) {
        line = read_dynamic_line(file);
        while (strlen(line) == 0) line = read_dynamic_line(file);

        if (!line) {
            printf("Ошибка: не удалось прочитать строку %d\n", i + 1);
            for (int k = 0; k < i; k++) free(matrix[k]);
            free(matrix);
            fclose(file);
            return 1;
        }

        char *ptr = line;
        char *token;

```

```

    for (int j = 0; j < cols; j++) {
        while (*ptr && isspace(*ptr)) ptr++; // Пропускаем пробельные символы

        // Считываем число
        token = ptr;
        while (*ptr && !isspace(*ptr)) ptr++;

        // Если пользователь ввел матрицу весов, то заменим веса на единицы, то есть на
        // обычные ребра (превратим матрицу весов в матрицу смежности)
        double token_double = (double)strtod(token, NULL);
        if (token_double == 0) matrix[i][j] = 0;
        else matrix[i][j] = 1;
    }
    free(line);
}

// Вывод матрицы смежности графа
printf("Матрица %dx%d:\n", rows, cols);
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        printf("%g ", matrix[i][j]);
    }
    printf("\n");
}

int n = rows;
topological_sort_dynamic(n, matrix);

for (int i = 0; i < rows; i++) free(matrix[i]);
free(matrix);
}

fclose(file);
return 0;
}

```

Примеры работы программы

Содержимое файла matrix.txt	Результат работы программы
<pre> 1 0 1 1 1 1 2 0 0 0 1 0 3 0 0 0 1 1 4 0 0 0 0 1 5 0 0 0 0</pre>	В какой массив записать матрицу? (1 - статический, 2 - динамический): Матрица 5x5: <pre> 0 1 1 1 1 0 0 0 1 0 0 0 0 1 1 0 0 0 0 1 0 0 0 0 0</pre> Топологический порядок вершин: [1, 3, 2, 4, 5]
<pre> 1 0 1 1 1 1 2 0 0 0 1 0 3 0 0 0 1 1 4 0 0 0 0 1 5 0 0 0 0</pre>	В какой массив записать матрицу? (1 - статический, 2 - динамический): Матрица 5x5: <pre> 0 1 1 1 1 0 0 0 1 0 0 0 0 1 1 0 0 0 0 1 0 0 0 0 0</pre> Топологический порядок вершин: [1, 3, 2, 4, 5]
<pre> 1 1 1 1 2 0 1 1 3 0 0 1</pre>	В какой массив записать матрицу? (1 - статический, 2 - динамический): Матрица 3x3: <pre> 1 1 1 0 1 1 0 0 1</pre> Ошибка: граф содержит циклы, топологическая сортировка невозможна
<pre> 1 0 0 0 2 1 0 0 3 1 1 0 4 </pre>	В какой массив записать матрицу? (1 - статический, 2 - динамический): Матрица 3x3: <pre> 0 0 0 1 0 0 1 1 0</pre> Топологический порядок вершин: [3, 2, 1]
<pre> 1 1 0 0 2 0 1 0 3 0 0 1 </pre>	В какой массив записать матрицу? (1 - статический, 2 - динамический): Матрица 3x3: <pre> 1 0 0 0 1 0 0 0 1</pre> Ошибка: граф содержит циклы, топологическая сортировка невозможна
Любой вид	В какой массив записать матрицу? (1 - статический, 2 - динамический): Ошибка: некорректный тип массива
1 Алгоритмы и структуры данных	В какой массив записать матрицу? (1 - статический, 2 - динамический): 1 Ошибка: некорректное значение 'Алгоритмы'
<pre> 1 0 0 0 2 3 4 1 00 0 5 6 7 8 1 1 0</pre>	В какой массив записать матрицу? (1 - статический, 2 - динамический): 1 Матрица 3x3: <pre> 0 0 0 1 0 0 1 1 0</pre> Топологический порядок вершин: [3, 2, 1]
<pre> 1 0 0 0 2 12 0 0 3 52 48 0</pre>	В какой массив записать матрицу? (1 - статический, 2 - динамический): 2 Матрица 3x3: <pre> 0 0 0 1 0 0 1 1 0</pre> Топологический порядок вершин: [3, 2, 1]

1	0 0 g	В какой массив записать матрицу? (1 - статический, 2 - динамический): 1 Ошибка: некорректное значение 'g'
2	12 jhgkjfgh 0	PS C:\Users\Анастасия> ./program
3	52 48 0	В какой массив записать матрицу? (1 - статический, 2 - динамический): 2 Ошибка: некорректное значение 'g'
1	0 0 1	В какой массив записать матрицу? (1 - статический, 2 - динамический): 1 Ошибка: разное количество элементов в строках
2	12	PS C:\Users\Анастасия> ./program
3	52 48 0	В какой массив записать матрицу? (1 - статический, 2 - динамический): 2 Ошибка: разное количество элементов в строках